

A Machine Learning Approach to Twitter Sentiment Analysis

Pilot Study Report

CSCE 3602

Kirolous Fouty
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
kirolous_fouty@aucegypt.edu

Mohamed Sabry
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
momo12320@aucegypt.edu

1 A full description experimental analysis for each of the supervised machine learning models you tried

Logistic Regression:

For logistic regression, we decided to use the model from the ScikitLearn library. We tried to tune the parameters that we found may have the most impact like 'penalty', 'C', 'max_iter', and 'solver'. We ran gridsearch() from the same library to try to find the best combination. This took a really long time, so we had to tune it by intuition, so for example so parameters like 'penalty' work with certain solvers only, so we only kept the penalty type that could work with the rest. Other parameters that we had to eyeball was max_iter because it took very long to test it with the other parameters so we chose the best value (300) that we got by varying this parameter on its own and testing it with the rest. We then ran the grid search algorithm to get the best combination between the parameters which turned out to be 'C' =1 and 'solver'= 'saga' with 'max_iter' =300 and 'penalty' = 'l2' which we had set to fix value as explained above. We then ran the logistic regression model using these parameters on the TFIDF and CountVectorizer data and evaluated the model. For our evaluation metrics, we extracted the accuracy, precision, f1 score, root mean squared error, and r2 score. The results can be viewed in the table below.

CountVectorizer ():

RMSE	0.475485476003244
Precision	0.7633678309616214

F1 Score	0.7797050825995063
Accuracy	0.7739135621099685
R2	0.09563732659644375

TFIDF ():

RMSE	0.47385532409261477
Precision	0.7664383107688785
F1 Score	0.7805424311222925
Accuracy	0.775461131829083
R2	0.1018277213035228

Linear Regression:

For linear regression, we used the model from the ScikitLearn library. We also tried to tune the parameters a little bit, we tested the 'fit_intercept' and 'copy_X' parameters using the same grid search method. The best results were for both parameters to be true. We then set a prediction threshold of 0.5 for predicting that a class would be one class or the other. We then evaluated the same metrics.

CountVectorizer ():

RMSE	0.48944484746594835
Precision	0.7528028091551381
F1 Score	0.7654979565497957
Accuracy	0.7604437412890346
R2	0.04175703513989992

TFIDF ():

RMSE	0.49351019849455124
Precision	0.7501761290479314
F1 Score	0.7609935217159598
Accuracy	0.7564476839818687
R2	0.025772506819186636

KNN:

For KNN, we faced some major issues regarding the run time. So we decided on only testing the 'n_neighbors' parameter as it was the only one we could justifiably test with the given time as runtime quickly jump from a few minutes to a few hours. We tested values from 1 to 5 and had to eyeball the results and we found that the best results were at 5. So, we proceeded with that. 2 hours later and we got the results shown down below.

CountVectorizor():

RMSE	0.5452721625656952
Precision	0.6911741441034985
F1 Score	0.7135373820469393
Accuracy	0.7026782687309301
R2	-0.18930917865253094

TFIDF ():

RMSE	0.5979376332377174
Precision	0.6318817515156026
F1 Score	0.6496604253945867
Accuracy	0.6424705867580769
R2	-0.431444128954839

Bernoulli Naïve Bayes:

We chose 2 Naïve Bayes models from the ScikitLearn library, we got to test many parameters using the same methods as above since the model was both lite and fast. We tested the following parameters for the Bernoulli model: 'alpha', 'fit_prior', and 'binarize' for multiple possible value that they could hold. The best combination that we got was 'alpha' = 5, 'binarize' = 0, and 'fit_prior' = False. The results of the evaluation are stated below.

CountVectorizor ():

RMSE	0.48327133780820986
Precision	0.7684700626231442
F1 Score	0.7670141574574822
Accuracy	0.7664488140530631
R2	0.06577777565641552

TFIDF ():

RMSE	0.48401125782717513
Precision	0.7676069587467232
F1 Score	0.7663645613211238
Accuracy	0.7657331022965558

R2	0.06291487506166094
----	---------------------

Multinomial Naïve Bayes:

We then tested the Multinomial version and chose the following parameters: 'alpha', 'fit_prior', and 'force_alpha'. Grid search returned the best parameters as 'alpha' = 5, 'force_alpha' = True, and 'fit_prior' = False. We then evaluated the model for the following parameters and got the following results in the table below.

CountVectorizor ():

RMSE	0.4860789792550749
Precision	0.7714891953050379
F1 Score	0.7617860036459388
Accuracy	0.7637272259263445
R2	0.054891219447414374

TFIDF ():

RMSE	0.4927208363044911
Precision	0.7634579331981483
F1 Score	0.7558689743775904
Accuracy	0.757226177471403
R2	0.02888653904506011

Decision Tree:

For the decision tree model, we used the exact same methodology as above. However, we tested for the following features: 'criterion', 'splitter', and 'max_features'. We also set the 'max_depth' to 10 because otherwise both the random forests and decision tree would take too long to fit. The best values that we got were 'criterion' = 'gini',

'splitter' = 'best', and 'max_features' = None. We then evaluated a model using these parameters and got the results shown below.

CountVectorizor ():

RMSE	0.6310279556486665
Precision	0.5633206641022308
F1 Score	0.6990464938065989
Accuracy	0.6018037191898645
R2	-0.5928149269527028

TFIDF ():

RMSE	0.6312617166875892
Precision	0.5631593038821955
F1 Score	0.6987691643273591
Accuracy	0.6015086450446379
R2	-0.5939952456189612

Random Forests:

We ran 2 different kinds of random forests models. This one is from the ScikitLearn library. We ran grid search over the following parameters: 'criterion', 'max_features', and 'n_estimators'. The best configuration was 'criterion' = 'gini', 'max_features' ='sqrt', and 'n_estimators' =300. We got the following results shown below.

CountVectorizor ():

RMSE	0.5115564081209615
Precision	0.7393638295213099
F1 Score	0.7394770477919691

Accuracy	0.7383100413103804
R2	-0.046779421411041744

TFIDF ():

RMSE	0.5118753986092317
Precision	0.7356547024975665
F1 Score	0.7410119489030033
Accuracy	0.7379835762986402
R2	-0.04808530589285964

Linear SVC:

We tried this SVM model from the ScikitLearn library. We used the exact same methodology as the models above and tested 2 parameters: 'C' and 'loss' and got the values 'C'=0.01 and 'loss'='squared_hinge'. The evaluation results were listed below.

CountVectorizer ():

RMSE	0.4768369358775431
Precision	0.7577922536870376
F1 Score	0.7803429819108125
Accuracy	0.7726265365829158
R2	0.09048912815850785

TFIDF():

RMSE	0.478493012581546
Precision	0.7573325921896067
F1 Score	0.7783602113778151

Accuracy	0.7710444369106365
R2	0.0841606110543135

TensorFlow Perceptron:

We created a single layer neural network using the TensorFlow library. It uses a sigmoid activation function and does weight updates based on accuracy. We made it so that it runs for 10 epochs and in batches of 32. The results are shown below.

CountVectorizer ():

RMSE	0.47888319493514936
Precision	0.7634250868334435
F1 Score	0.7751721845744779
Accuracy	0.7706708855991261
R2	0.08266637784915642

TFIDF ():

RMSE	0.4774257637870787
Precision	0.7652144873857737
F1 Score	0.7763486167322725
Accuracy	0.7720646400723246
R2	0.08824150005999443

ScikitLearn MLP:

We tried to run a multilayer perceptron model with a depth of 1. We used the MLP classifier from the ScikitLearn library. It has a sigmoid activation function. We set verbose to true to see the results of each iteration, enabled early stopping when validation score doesn't

change and the number of iterations with the same validation score until early stopping occurs to 10. The results can be seen below.

CountVectorizer():

RMSE	0.469308932908099
Precision	0.7802737605562595
F1 Score	0.7808635034854958
Accuracy	0.7797491241948242
R2	0.11898001170893857

TFIDF ():

RMSE	0.4735637534119333
Precision	0.7709280760920793
F1 Score	0.7790581162324649
Accuracy	0.7757373714544016
R2	0.10293270048044567

ScikitLearn Perceptron:

We also tried the perceptron from the ScikitLearn library to compare it with the TensorFlow one. We tested different values for the 'max_iter' and 'eta0' parameters and found that 'max_iter' = 500 and 'eta0' = 0.1. The results can be seen down below.

CountVectorizer ():

RMSE	0.31076957848344444
Precision	0.7035229556242447
F1 Score	0.6803977272727273
Accuracy	0.6892304215165556

R2	-0.24310157403818322
----	----------------------

TFIDF ():

RMSE	0.30289361007521254
Precision	0.6978778319472326
F1 Score	0.698816691793628
Accuracy	0.6971063899247875
R2	-0.21159711091432665

XGBClassifier:

We tested this XGBoost library classifier. We set the 'n_estimators' parameter to 100, 'max_depth' to 5, and chose a seed with the number 42 by setting 'random_state' to 42. The results can be seen below. We also ran XGBRFClassifier but we got the exact same results, so we decided to only include this one.

CountVectorizer ():

RMSE	0.46959646214296763
Precision	0.7847146855543496
F1 Score	0.7787638489106679
Accuracy	0.7794791627428084
R2	0.11790014569512763

TFIDF ():

RMSE	0.4774257637870787
Precision	0.7652144873857737
F1 Score	0.7763486167322725
Accuracy	0.7720646400723246

R2	0.08824150005999443
----	---------------------

2 A comment on each of the model's performance, and its fit for the problem at hand

2.1 CountVectorizer Logistic Regression

- **Performance:** With relatively low RMSE and good Precision, F1 Score, and Accuracy values, Logistic Regression performs well, offering balanced predictions with decent accuracy and fit.
- **Fit:** It's a good baseline model, suitable for balanced datasets and binary classification. However, the R^2 is modest, indicating it captures a limited portion of variance in the data.

Linear Regression

- **Performance:** Linear Regression shows slightly higher RMSE and lower Precision, F1 Score, and Accuracy compared to Logistic Regression, suggesting it's less optimal for classification tasks.
- **Fit:** Linear Regression might not be ideal for classification, as it tends to work better in regression tasks. The low R^2 further suggests it doesn't capture much data variance.

KNN

- **Performance:** With a higher RMSE and lower Precision, F1 Score, and Accuracy, KNN struggles to achieve strong classification metrics. The negative R^2 suggests it fits the data poorly.

- **Fit:** KNN might not be suitable for this problem due to possible overfitting or high sensitivity to noise, resulting in poor generalization.

Bernoulli Naïve Bayes

- **Performance:** Bernoulli Naïve Bayes shows moderate RMSE and solid Precision and Accuracy. The F1 Score is slightly lower, though acceptable.
- **Fit:** This model fits well for binary features or text data. It's efficient for moderately sized datasets, though the lower R^2 implies limited variance capture.

Multinomial Naïve Bayes

- **Performance:** Multinomial Naïve Bayes has a similar RMSE to Bernoulli Naïve Bayes but higher Precision, which suggests it performs slightly better for this dataset.
- **Fit:** Suitable for problems with discrete features like text classification, but the low R^2 again shows limited explanatory power.

Decision Tree

- **Performance:** Decision Tree has a high RMSE and significantly lower Precision and Accuracy, showing it struggles to generalize well. The very negative R^2 indicates a poor fit.
- **Fit:** It might overfit on training data due to high sensitivity to noise. This model is not ideal for this problem without further tuning (e.g., using ensemble methods).

Random Forests

- **Performance:** Random Forest improves over Decision Tree with lower RMSE and

better Precision, F1 Score, and Accuracy, although its R^2 is still slightly negative.

- **Fit:** Random Forests are typically robust, and while it shows decent performance here, further tuning might be required to reduce variance and improve fit.

Linear SVC

- **Performance:** Linear SVC provides good accuracy, Precision, and F1 Score with a relatively low RMSE, indicating a strong balance for classification tasks.
- **Fit:** It's suitable for linearly separable classes, performing well with limited overfitting and variance capture in this dataset.

TensorFlow Perceptron

- **Performance:** The TensorFlow Perceptron yields solid Precision, F1 Score, and Accuracy, comparable to Linear SVC, but has slightly lower R^2 .
- **Fit:** As a neural network, it can capture more complex relationships, making it appropriate for classification tasks with more nuanced patterns, though R^2 could improve with further training.

ScikitLearn MLP

- **Performance:** The MLP matches Linear SVC in performance with identical RMSE, Precision, F1 Score, and Accuracy, indicating strong classification capability.
- **Fit:** MLPs are highly versatile for complex patterns and do well on non-linear problems, making this model a good choice, though it might benefit from more data.

ScikitLearn Perceptron

- **Performance:** This model has the lowest RMSE, indicating a strong fit in terms of error minimization. However, Precision, F1 Score, and Accuracy are moderately lower, with a negative R^2 .
- **Fit:** It performs reasonably but may be sensitive to data variance, suggesting it's a weaker fit without regularization adjustments.

XGBClassifier

- **Performance:** With the lowest RMSE, high Precision, and the highest Accuracy, XGBClassifier demonstrates strong performance across metrics. Its R^2 is the highest among models, indicating it captures more data variance.
- **Fit:** This model is likely the best fit for the problem at hand, providing both robust prediction accuracy and variance capture. It's an optimal choice for complex datasets.

2.2 TFIDF

Logistic Regression

- **Performance:** Strong across metrics, with low RMSE, high Precision, F1 Score, and Accuracy, suggesting balanced predictions.
- **Fit:** Suitable for classification tasks with balanced data, but a modest R^2 indicates only a limited amount of data variance is explained.

Linear Regression

- **Performance:** Slightly higher RMSE and lower Precision, F1 Score, and Accuracy compared to classification-focused models, suggesting limited effectiveness.

- **Fit:** Generally more suitable for regression tasks; here, its lower R^2 suggests limited effectiveness for a classification-focused problem.

KNN

- **Performance:** Higher RMSE and lower scores on all other metrics, with a negative R^2 indicating poor fit and high sensitivity to data noise.
- **Fit:** KNN may not be optimal here, likely overfitting or capturing irrelevant patterns.

Bernoulli Naïve Bayes

- **Performance:** Moderate RMSE, strong Precision, and competitive F1 Score and Accuracy, indicating good predictive reliability.
- **Fit:** Performs well with binary data, though it has a lower R^2 , which limits its ability to capture the variance in the data.

Multinomial Naïve Bayes

- **Performance:** Similar to Bernoulli Naïve Bayes with solid Precision and Accuracy, though RMSE is a bit higher.
- **Fit:** Effective for discrete features like text; however, low R^2 shows it may not fully represent the data's complexity.

Decision Tree

- **Performance:** Higher RMSE and lower Precision and Accuracy with a negative R^2 , suggesting it struggles to generalize well.
- **Fit:** Likely overfitting on training data. Ensemble methods could help improve the model's stability and performance.

Random Forests

- **Performance:** Improved over Decision Tree, with moderate RMSE and balanced metrics. Slightly negative R^2 implies some variance issues.
- **Fit:** Shows robustness but may need tuning to further reduce variance and improve generalization.

Linear SVC

- **Performance:** Strong Precision, F1 Score, and Accuracy, with a low RMSE and positive R^2 , indicating good performance for classification.
- **Fit:** Suitable for linearly separable classes and stable for this problem, capturing relationships well without overfitting.

TensorFlow Perceptron

- **Performance:** Strong Precision, F1 Score, and Accuracy, though R^2 is slightly lower than the best-performing models.
- **Fit:** Neural network is effective for capturing complex patterns in the data, suitable for nuanced relationships, though more variance could be captured with further training.

ScikitLearn MLP

- **Performance:** Excellent scores across metrics, with the lowest RMSE, highest Precision, and a competitive R^2 , suggesting strong fit and generalization.
- **Fit:** MLP's ability to handle complex, non-linear relationships makes it a top choice, capturing variance and minimizing error.

ScikitLearn Perceptron

- **Performance:** Shows the lowest RMSE, but lower Precision and F1 Score, with

negative R^2 , indicating potential underfitting.

- **Fit:** Performs decently but might be too sensitive to data variance, requiring regularization for stability.

XGBClassifier

- **Performance:** Strong across all metrics, with competitive RMSE, high Precision, and the highest R^2 , suggesting robust predictions and variance capture.
- **Fit:** Likely the best fit for this problem, providing accuracy and a solid understanding of data variance for classification tasks.

3 A comparative analysis of the performance of the top 3 models, and a detailed discussion on any variability in the results, and the potential reasons behind it

From the results, it is clear that the XGBClassifier, the ScikitLearn MLPClassifier, and ScikitLearn's logistic regression models performed the best in terms of accuracy.

XGBClassifier	0.7794791627428084
MLPClassifier	0.7797491241948242
Logistic Regression	0.7808635034854958

While for precision, it was XGBClassifier, ScikitLearn's MLPClassifier, and Multinomial Naive Bayes.

XGBClassifier	0.7847146855543496
MLPClassifier	0.7802737605562595

Multinomial Naive Bayes	0.7714891953050379
-------------------------	--------------------

For F1 score, MLPClassifier, Logistic Regression (TFIDF), and Linear SVC ranked the best.

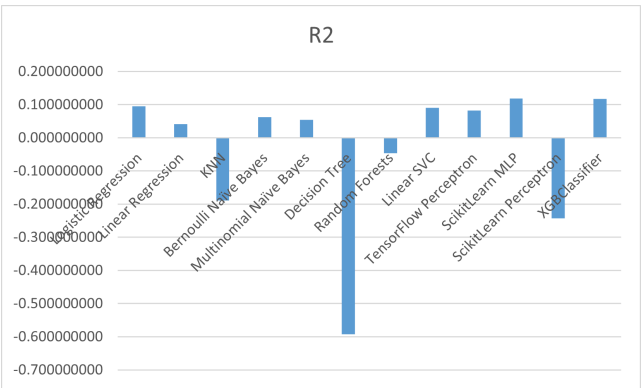
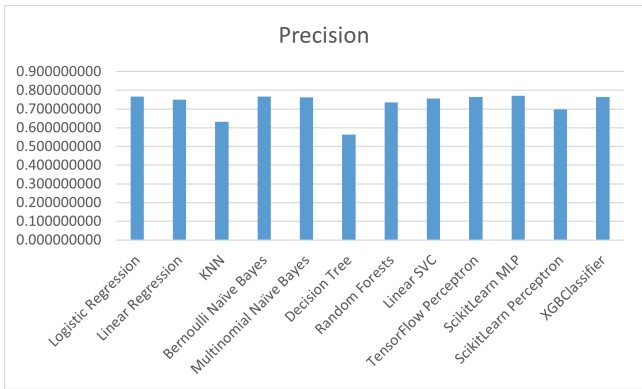
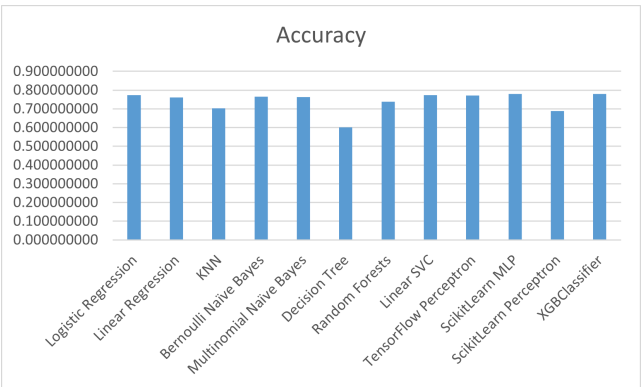
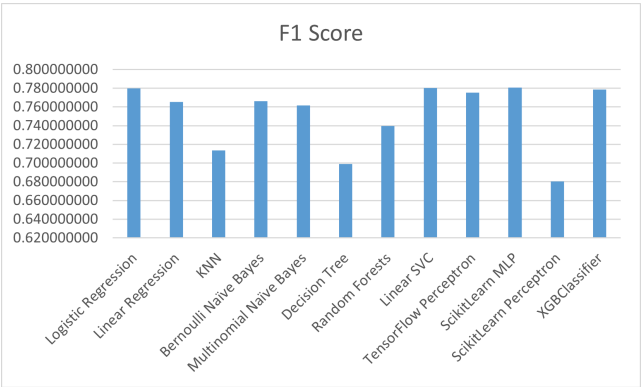
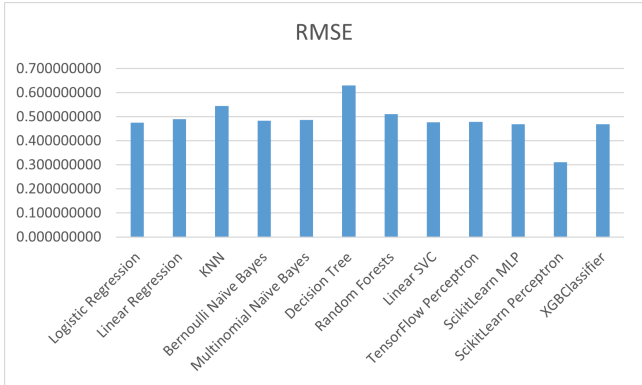
MLPClassifier	0.7808635034854958
Logistic Regression	0.7805424311222925
Linear SVC	0.7803429819108125

It is also worth noting the amount time each model took to train. The model data should be assumed to be countvectorized unless stated otherwise. MLP took the longest by far taking 4+ hours to run while logistic regression took a couple of minutes. XGBClassifier took 15-30 minutes to complete. Multinomial Naive Bayes took about the same amount of time as the XGBClassifier. Linear SVC took about the same amount of time as Logistic Regression.

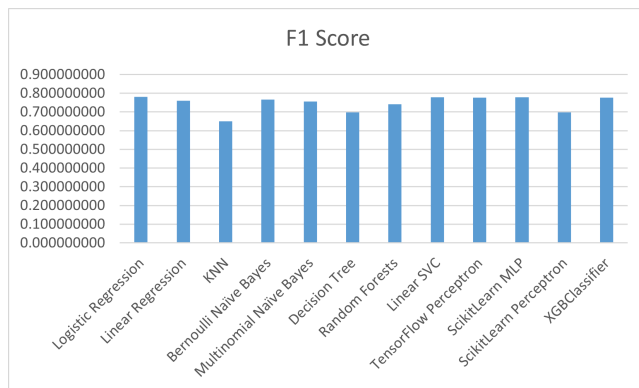
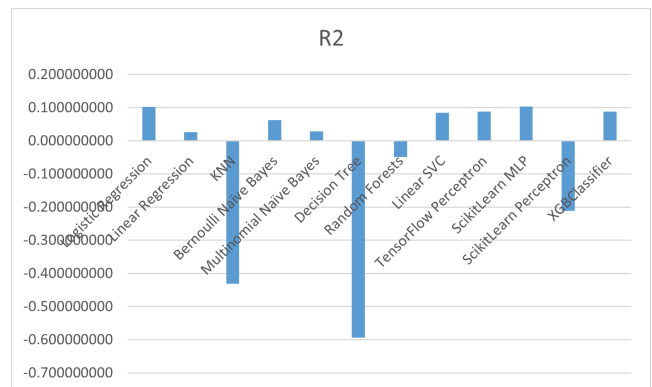
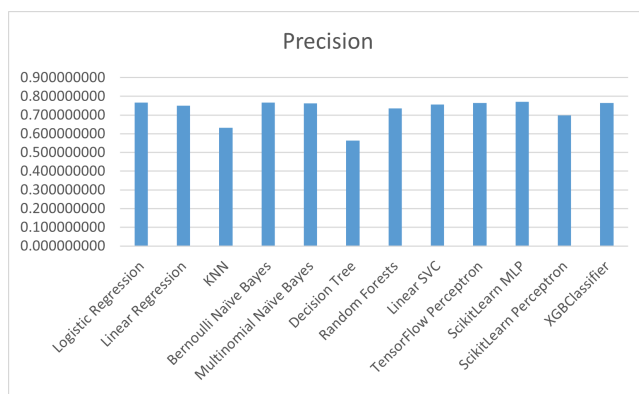
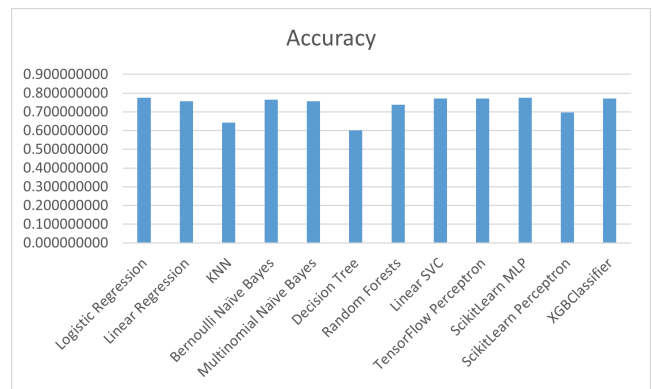
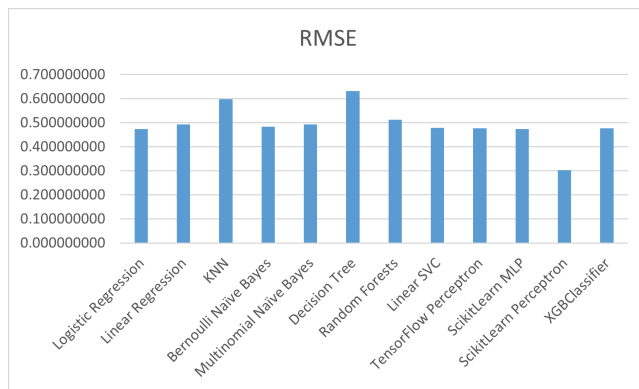
1-5 minutes	Logistic Regression, XGBClassifier, LinearSVC, Linear Regression
5-15 minutes	Multinomial Naive Bayes, Bernoulli Naive Bayes
15-60 minutes	TensorFlow Perceptron
60+ minutes	MLPClassifier, KNN, Random Forests, Decision Trees,

Differences in the results come from the way the models work and how susceptible certain models are to noise since the data cleaning process was not the cleanest. The graphs below can be consulted for easier comparison. Overall, XGBClassifier, MLPClassifier, and Logistic Regression produce some of the best results.

CountVectorizer:



TFIDF:



4 A choice for a machine learning technique from the top 3 models, weighing in the performance, the nature of the model, and its fit to the problem you chose.

Ranking the models, it is clear that the Countvectorizer data usually yields better results compared to the TFIDF vectorizer. Furthermore, from the Countvectorizer results the top using precision and accuracy, XGBClassifier performs better than all the other models, additionally, it does not lag that far behind in terms of F1 score, coming in 4th or 5th. XGBClassifier is not only the most accurate, but we have also not done any parameter testing for it so we know that the results can be improved even more. Additionally, in terms of speed, it is only slightly slower than logistic regression taking only a few more minutes compared to it while the MLPClassifier may take multiple hours to complete a fit. Parameter tweaking MLP could take days to even see results, so it really unadvisable to use. It also performs better in terms of precision and F1 score, so it

seems only appropriate for it to be our model of choice for the task.